

Class 5 - HTTP, Routing & Navigation

Table of Contents

- ☐ Fetching Data with Native fetch
- ☐ Fetching Data with Observables & HttpClient
- ☐ Proxy Configuration
- ☐ Environment Files
- ☐ Adding Routes
- ☐ Redirect Routes
- ☐ Linking Routes
- ☐ Route Parameters
- ☐ Query Parameters

1. Fetching Data with Native fetch

The browser has a built-in `fetch` function to call APIs.

```
async fetchBooks() {  
  const response = await fetch('http://localhost:3000/api/boo  
ks');  
  const data = await response.json();  
  this.products = data;  
}
```

How it works

Step	What happens
<code>fetch(url)</code>	Sends HTTP GET request, returns a Promise

Step	What happens
<code>await</code>	Waits for the response to arrive
<code>.json()</code>	Parses the response body as JSON

2. Fetching Data with Observables & HttpClient

What is an Observable?

An **Observable** is a stream of data over time. Think of it like a water tap:

- **Promise** — one-time delivery. Order a pizza, it arrives once.
- **Observable** — continuous stream. A tap you can turn on, listen to, and turn off.

Calling an API with HttpClient

In the service (`cart.ts`):

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class CartService {
  constructor(private http: HttpClient) {}

  getProducts(): Observable<IProduct[]> {
    return this.http.get<IProduct[]>('http://localhost:3000/api/books');
  }
}
```

Subscribing in a component

In `catalog.ts` :

```
fetchBooks() {
  this.cartService.getProducts().subscribe((products) => {
    this.products = products;
  });
}
```

Observable vs fetch — side by side

```
// Native fetch (Promise)
async fetchBooks() {
  const response = await fetch('http://localhost:3000/api/books');
  this.products = await response.json();
}

// HttpClient (Observable)
fetchBooks() {
  this.cartService.getProducts().subscribe((products) => {
    this.products = products;
  });
}
```

Key rule

`http.get()` does **nothing** until you call `.subscribe()`. The Observable is lazy — it only runs when someone is listening.

3. Proxy Configuration

The Problem — CORS

In development, your Angular app runs on port **4200** and your backend runs on port **3000**. The browser blocks requests between different origins — this is called **CORS** (Cross-Origin Resource Sharing).

```
Browser → localhost:4200 (Angular)
Browser → localhost:3000 (Backend API) ← BLOCKED by browser
```

The Solution — Proxy

A proxy makes Angular's dev server forward API requests to the backend on your behalf. The browser only talks to port 4200, so there is no cross-origin issue.

```
Browser → localhost:4200/api/books
        → Angular Dev Server (proxy)
          → localhost:3000/api/books ← server-to-server, no CORS
```

Step 1 — Create `proxy.conf.json`

```
{
  "/api": {
    "target": "http://localhost:3000"
  }
}
```

This says: *"any request starting with `/api` should be forwarded to `http://localhost:3000`"*

Step 2 — Register in `angular.json`

```
"serve": {
  "options": {
    "proxyConfig": "proxy.conf.json"
  }
}
```

Step 3 — Use relative URLs in your code

```
// Before proxy – hardcoded full URL
this.http.get('http://localhost:3000/api/books')
```

```
// After proxy – clean relative URL
this.http.get('/api/books')
```

Now your code no longer has a hardcoded port number. The proxy handles where the request goes.

4. Environment Files

The Problem

Your API URL changes between environments:

Environment	URL
Development	<code>http://localhost:3000</code>
Production	<code>https://api.myprogramminghub.com</code>

You don't want to manually change URLs before every deployment.

The Solution — Environment Files

Angular provides two files:

```
src/environments/
  environment.ts          ← used in production build
  environment.development.ts ← used in development build
```

`environment.development.ts`

```
export const environment = {
  baseUrl: 'http://localhost:3000'
};
```

`environment.ts` (production)

```
export const environment = {
  baseUrl: 'https://api.myprogramminghub.com'
};
```

Using the environment in a service

```
import { environment } from '../environments/environment';

@Injectable({ providedIn: 'root' })
export class CartService {
  private baseUrl = environment.baseUrl;

  getProducts(): Observable<IProduct[]> {
    return this.http.get<IProduct[]>(`${this.baseUrl}/api/books`);
  }
}
```

Always import from `environment.ts` — Angular automatically swaps the file at build time.

How Angular swaps the file — `angular.json`

```
"fileReplacements": [
  {
    "replace": "src/environments/environment.ts",
    "with": "src/environments/environment.development.ts"
  }
]
```

When you run `ng serve` (development), Angular replaces `environment.ts` with `environment.development.ts` automatically.

```
ng serve          → uses environment.development.ts
```

```
ng build → uses environment.ts (production)
```

5. Adding Routes

What is Routing?

Routing lets you show different components based on the URL — without reloading the page. This is what makes Angular a **Single Page Application (SPA)**.

```
/home → shows Home component  
/catalog → shows Catalog component
```

Step 1 — Define routes in `app.routes.ts`

```
import { Routes } from '@angular/router';  
import { Home } from './home/home';  
import { Catalog } from './catalog/catalog';  
  
export const routes: Routes = [  
  { path: 'home', component: Home },  
  { path: 'catalog', component: Catalog }  
];
```

Step 2 — Register routes in `app.config.ts`

```
import { provideRouter } from '@angular/router';  
import { routes } from './app.routes';  
  
export const appConfig: ApplicationConfig = {  
  providers: [provideRouter(routes)]  
};
```

Step 3 — Add `<router-outlet>` in `app.html`

```
<book-header></book-header>
<router-outlet></router-outlet>
```

`<router-outlet>` is the placeholder where Angular renders the matched component.

```
URL: /home    → <router-outlet> renders <book-home>
URL: /catalog → <router-outlet> renders <book-catalog>
```

6. Redirect Routes

A redirect route automatically sends the user from one URL to another.

Basic redirect

```
{ path: '', redirectTo: '/home', pathMatch: 'full' }
```

Property	Value	Meaning
<code>path</code>	<code>''</code>	matches the empty URL <code>/</code>
<code>redirectTo</code>	<code>'/home'</code>	sends the user to <code>/home</code>
<code>pathMatch</code>	<code>'full'</code>	only match when URL is exactly empty

`pathMatch: 'full'` vs `pathMatch: 'prefix'`

	<code>'full'</code>	<code>'prefix'</code>
Matches	Exact URL only	URL starts with path
Use for empty path?	Yes 	No  — causes infinite redirect

Why `prefix` causes infinite redirect on empty path:

```
pathMatch: 'prefix' → '' is a prefix of EVERY URL (including '/home')
/home → redirect to /home → redirect to /home → ☀ infinite loop
```

Always use `pathMatch: 'full'` for empty path redirects.

Complete routes example

```
export const routes: Routes = [
  { path: '',          redirectTo: '/home', pathMatch: 'full' },
  { path: 'home',      component: Home },
  { path: 'catalog',   component: Catalog }
];
```

7. Linking Routes

routerLink — navigate without page reload

Use **routerLink** instead of **href**. **href** reloads the whole page. **routerLink** navigates inside Angular without a reload.

```
<!-- Wrong – full page reload -->
<a href="/catalog">Catalog</a>

<!-- Correct – Angular navigation, no reload -->
<a routerLink="/catalog">Catalog</a>
```

Array syntax

```
<a [routerLink]="['/catalog']">Catalog</a>
```

Useful when building paths dynamically:

```
<a [routerLink]="['/catalog', category]">{{ category }}</a>
```

Absolute vs Relative links

```
<!-- Absolute – always resolves from root / -->
<a [routerLink]="['/catalog']">Catalog</a>
```

```
<!-- Relative – resolves from the current route's URL -->
<a [routerLink]="['catalog']">Catalog</a>
```

routerLinkActive — highlight the active link

Angular automatically adds a CSS class when the route is active:

```
<a routerLink="/home" routerLinkActive="active">Home</a>
<a routerLink="/catalog" routerLinkActive="active">Catalog</a>
>
```

```
.active {
  color: #8B2635;
  font-weight: bold;
  background-color: #f5e6e8;
  padding: 4px 10px;
  border-radius: 6px;
}
```

Importing RouterLink in a component

Since Angular uses standalone components, you must import `RouterLink` and `RouterLinkActive` explicitly:

```
import { RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  imports: [RouterLink, RouterLinkActive],
  ...
})
export class Header {}
```

8. Route Parameters

Route parameters let you pass data as part of the URL path.

```
/catalog/Programming    → filter = 'Programming'  
/catalog/Algorithms     → filter = 'Algorithms'
```

Step 1 — Define the parameter in the route

```
{ path: 'catalog/:filter', component: Catalog }
```

`:filter` is the parameter name. It matches any value in that URL position.

Step 2 — Read the parameter in the component

```
import { ActivatedRoute } from '@angular/router';  
  
export class Catalog implements OnInit {  
  productFilter = '';  
  
  constructor(private activatedRoute: ActivatedRoute) {}  
  
  ngOnInit() {  
    this.activatedRoute.paramMap.subscribe((paramMap) => {  
      this.productFilter = paramMap.get('filter') ?? '';  
    });  
  }  
}
```

Step 3 — Link with a route parameter

```
<a [routerLink]="['/catalog', 'Programming']">Programming</a>  
<a [routerLink]="['/catalog', 'Algorithms']">Algorithms</a>
```

This generates `/catalog/Programming` and `/catalog/Algorithms`.

`paramMap.get()` returns `string | null`

If the parameter is missing from the URL, `.get()` returns `null`. Use `?? (nullish coalescing operator)` to provide a fallback:

```
this.productFilter = paramMap.get('filter') ?? '';  
//                                     ^^ if null, use  
empty string
```

9. Query Parameters

Query parameters pass optional data after a `?` in the URL.

```
/catalog?filter=Algorithms  
/catalog?filter=Programming&page=2
```

Difference from route parameters

	Route Param	Query Param
URL	<code>/catalog/Programming</code>	<code>/catalog?filter=Programming</code>
Required?	Yes (part of path)	No (optional)
Route definition	<code>path: 'catalog/:filter'</code>	<code>path: 'catalog'</code> (no change)

Step 1 — Link with query parameters

```
<a [routerLink]="['/catalog']" [queryParams]="{ filter: 'Algo  
rithms' }">  
  Algorithms  
</a>
```

`[queryParams]` requires binding brackets `[]` and an object value — not a plain string.

Step 2 — Read query parameters in the component

```
ngOnInit() {
  this.activatedRoute.queryParamMap.subscribe((paramMap) => {
    this.productFilter = paramMap.get('filter') ?? '';
  });
}
```

Use `queryParamMap` (not `paramMap`) for query parameters.

Full example from our app

home.html — linking with query param:

```
<a [routerLink]="['/catalog']" [queryParams]="{ filter: 'Algorithms' }">
  
  <div>ALGORITHMS</div>
</a>
```

catalog.ts — reading the query param:

```
ngOnInit() {
  this.activatedRoute.queryParamMap.subscribe((paramMap) => {
    this.productFilter = paramMap.get('filter') ?? '';
  });
  this.fetchBooks();
}
```

Navigating to `/catalog?filter=Algorithms` automatically filters the product list to show only Algorithms books.

Interview Question:

HTTP & Observables

Q1. What is the difference between a Promise and an Observable?

> A Promise resolves once and is eager (runs immediately). An Observable is a stream that can emit multiple values over time and is lazy — it only runs when you `.subscribe()`. Observables can also be cancelled.

Q2. What happens if you call `http.get()` but never `.subscribe()` ?

> Nothing. Observables are lazy — the HTTP request is never made until someone subscribes.

Q3. What is CORS and how does a proxy solve it?

> CORS (Cross-Origin Resource Sharing) is a browser security rule that blocks requests between different origins (e.g. port 4200 → port 3000). A dev proxy forwards requests from Angular's server to the backend server-to-server, which the browser never sees as cross-origin.

Environment & Proxy

Q4. What is the purpose of `environment.ts` and `environment.development.ts` ?

> They hold environment-specific configuration (like API base URLs). Angular swaps the file automatically at build time — `environment.development.ts` is used during `ng serve` and `environment.ts` is used for production builds.

Q5. Where do you register the proxy config in Angular?

> In `angular.json` under `serve.options.proxyConfig`. You point it to a `proxy.conf.json` file that maps URL prefixes to backend targets.

Routing

Q6. What is `<router-outlet>` and why is it needed?

> It is a placeholder in the template where Angular renders the component that matches the current URL. Without it, routed components have nowhere to display.

Q7. What is the difference between `pathMatch: 'full'` and `pathMatch: 'prefix'` ?

> `'full'` matches only when the entire URL equals the path. `'prefix'` matches when the URL starts with the path. For empty path redirects (`path: ''`), always use `'full'` — otherwise every URL matches and causes an infinite redirect loop.

Q8. Why does an empty path redirect cause an infinite loop with `pathMatch: 'prefix'` ?

> Because `''` (empty string) is a prefix of every URL, including the redirect destination like `/home`. So `/home` triggers the redirect again → `/home` → infinite loop.

Q9. What is the difference between `routerLink` and `href` ?

> `href` causes a full page reload — the entire Angular app restarts. `routerLink` navigates inside Angular without a reload, preserving application state and giving a faster user experience.

Q10. What is the difference between a route parameter and a query parameter?

> A route parameter is part of the URL path (`/catalog/Programming`) — it is required and defined in the route as `:paramName`. A query parameter comes after `?` (`/catalog?filter=Programming`) — it is optional and does not require changes to the route definition.

Q11. What does the `??` operator do?

> The nullish coalescing operator returns the right-hand value only when the left side is `null` or `undefined`. Unlike `||`, it does not treat `0`, `false`, or `''` as fallback triggers — only true null/undefined values.

Q14. Why is `routerLink="/catalog"` safer than `routerLink="catalog"` ?

> A relative link resolves against the current active route. A component inside `<router-outlet>` at `/home` would resolve `catalog` to `/home/catalog` — wrong. An absolute path (`/catalog`) always resolves from the root regardless of where the component is rendered.